3.3 Vista de Dominio — Diagrama de Clases UML

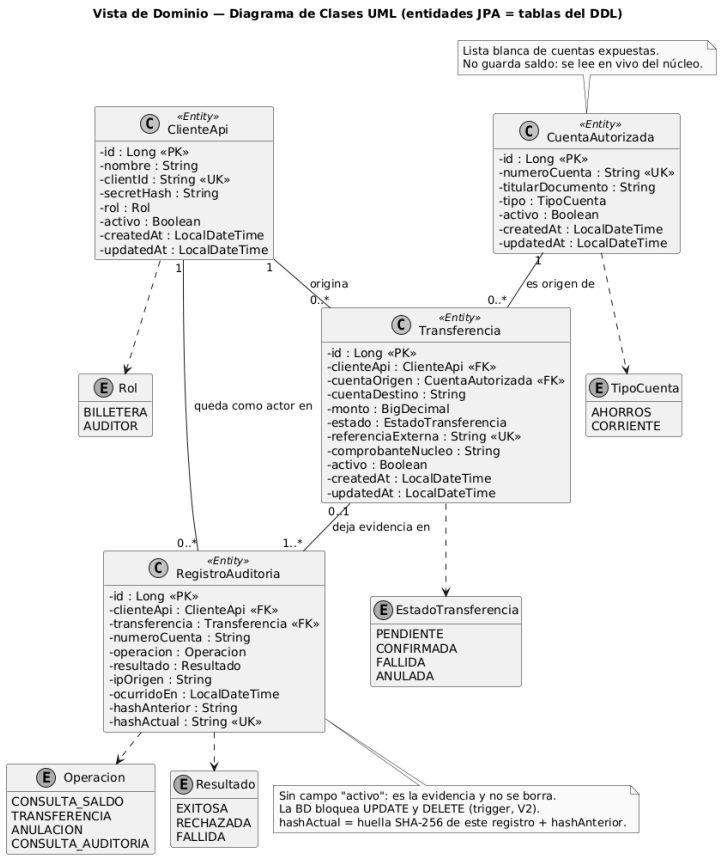


Figura 3. Diagrama de clases: entidades JPA, atributos tipados, llaves y multiplicidades.

Leyenda

- «PK» llave primaria, «FK» llave foránea, «UK» valor único. «Entity» = clase que JPA guarda como tabla. E = enumeración (valores permitidos).
- Números en las líneas: multiplicidad (1 = uno, 0..* = cero o muchos, 0..1 = cero o uno, 1..* = uno o muchos).

Lectura en 30 segundos

Cuatro entidades. Transferencia es la principal (HU-01): la origina un ClienteApi (la billetera) desde una CuentaAutorizada. Cada transferencia deja uno o más RegistroAuditoria, y cada registro guarda quién (clienteApi), qué (operacion), sobre qué cuenta, con qué resultado y cuándo. Dos decisiones para defender: CuentaAutorizada no guarda saldo, porque el saldo siempre se lee del núcleo; y RegistroAuditoria no tiene campo activo, porque es la evidencia y no se puede borrar de ninguna forma.

Correspondencia clase ↔ tabla

Clase (Java)	Tabla (PostgreSQL)	Borrado	Aparece en la secuencia (3.4)
ClienteApi	cliente_api	Lógico (activo)	Se identifica en FiltroJwt
CuentaAutorizada	cuenta_autorizada	Lógico (activo)	Es la cuenta origen de la transferencia (FK)
Transferencia	transferencia	Lógico (activo + ANULADA)	INSERT y UPDATE (pasos 9–10, 14–15, 22–23)
RegistroAuditoria	registro_auditoria	Ninguno (bloqueado por trigger)	INSERT bitácora (pasos 16–17, 24–25)

3.4 Vista Dinámica — Diagrama de Secuencia UML (HU-01)

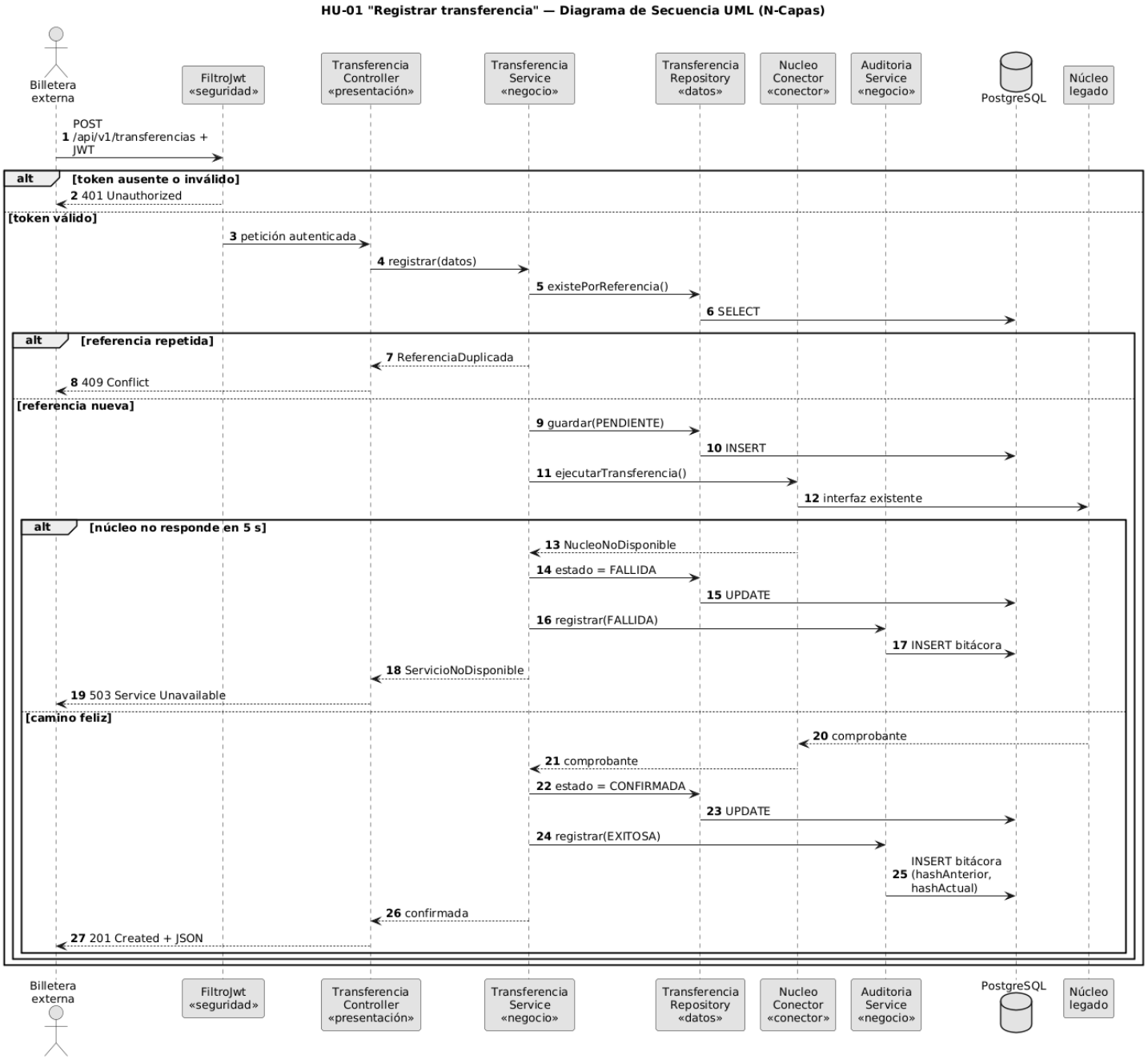


Figura 4. Secuencia de HU-01 "Registrar transferencia": camino feliz y tres flujos alternos (401, 409, 503).

Leyenda: flecha continua = llamada; flecha punteada = respuesta; recuadro alt = alternativas según una condición. Los pasos están numerados.

Lectura en 30 segundos: la petición atraviesa las capas en orden. Si no trae token válido, la seguridad responde 401 sin llegar al negocio. Si la referencia ya existe, el servicio responde 409 y no repite la operación. Si la referencia es nueva, se guarda como PENDIENTE y se llama al núcleo a través del conector: si el núcleo no responde en 5 s, la transferencia queda FALLIDA, se registra en la bitácora y se responde 503; si confirma, queda CONFIRMADA, se registra en la bitácora y se responde 201. En ambos casos queda evidencia, que es lo que pide el auditor.

3.5 Vista de Despliegue

Se entrega en la Clase 5 (24/10/2026), junto con el ADR y el arranque del corte vertical.

SECCIÓN 5.2: SCRIPT SQL DDL / MIGRACIONES

El esquema se divide en dos migraciones de Flyway, guardadas en db/migrations/ del repositorio. Se ejecutaron sobre PostgreSQL 16 y se verificaron con el script db/pruebas/verificar_restricciones.sql (resultado al final de esta sección).

db/migrations/V1__esquema_inicial.sql

```
-- ========================================================================
-- SITRA — Servicio de Integración y Trazabilidad (Caso 5, cooperativa)
-- V1: esquema inicial — PostgreSQL 16
-- Requisitos evaluables: PK en toda tabla, FK con REFERENCES, CHECK,
-- UNIQUE, borrado lógico (activo) en toda entidad de negocio e índices.
-- ========================================================================

-- Sistemas y personas autorizadas a usar SITRA (la billetera y el auditor).
CREATE TABLE cliente_api (
    id            BIGSERIAL     PRIMARY KEY,
    nombre        VARCHAR(120)  NOT NULL,
    client_id     VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash   VARCHAR(255)  NOT NULL,                -- BCrypt, nunca el secreto en claro
    rol           VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA'
                  CHECK (rol IN ('BILLETERA', 'AUDITOR')),
    activo        BOOLEAN       NOT NULL DEFAULT TRUE,    -- borrado lógico
    created_at    TIMESTAMP     NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMP     NOT NULL DEFAULT NOW()
);

-- Lista blanca: únicas cuentas que se pueden exponer a la billetera (RF-07).
-- NO guarda saldo: el saldo siempre se lee en vivo del núcleo (RF-01).
CREATE TABLE cuenta_autorizada (
    id                 BIGSERIAL    PRIMARY KEY,
    numero_cuenta      VARCHAR(30)  NOT NULL UNIQUE,
    titular_documento  VARCHAR(30)  NOT NULL,
    tipo               VARCHAR(20)  NOT NULL
                       CHECK (tipo IN ('AHORROS', 'CORRIENTE')),
    activo             BOOLEAN      NOT NULL DEFAULT TRUE, -- borrado lógico
    created_at         TIMESTAMP    NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMP    NOT NULL DEFAULT NOW()
);

-- Entidad principal de la HU-01 y del corte vertical.
CREATE TABLE transferencia (
    id                  BIGSERIAL      PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL REFERENCES cliente_api(id),
    cuenta_origen_id    BIGINT         NOT NULL REFERENCES cuenta_autorizada(id),
    cuenta_destino      VARCHAR(30)    NOT NULL,
    monto               NUMERIC(14,2)  NOT NULL CHECK (monto > 0),
    estado              VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE'
                        CHECK (estado IN ('PENDIENTE', 'CONFIRMADA', 'FALLIDA', 'ANULADA')),
    referencia_externa  VARCHAR(80)    NOT NULL UNIQUE,   -- evita duplicados (RF-08, 409)
    comprobante_nucleo  VARCHAR(80),                      -- lo devuelve el núcleo al confirmar
    activo              BOOLEAN        NOT NULL DEFAULT TRUE, -- borrado lógico (RF-10)
    created_at          TIMESTAMP      NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMP      NOT NULL DEFAULT NOW()
);

-- Bitácora de auditoría (RF-03, RF-05, AC-01 Seguridad).
-- No tiene campo "activo": es la evidencia; no se borra ni lógica ni físicamente.
CREATE TABLE registro_auditoria (
    id                BIGSERIAL    PRIMARY KEY,
    cliente_api_id    BIGINT       NOT NULL REFERENCES cliente_api(id),   -- quién
    transferencia_id  BIGINT       REFERENCES transferencia(id),          -- vacío en consultas de saldo
    numero_cuenta     VARCHAR(30)  NOT NULL,                              -- sobre qué cuenta
    operacion         VARCHAR(30)  NOT NULL                               -- qué
                      CHECK (operacion IN ('CONSULTA_SALDO', 'TRANSFERENCIA', 'ANULACION',
'CONSULTA_AUDITORIA')),
    resultado         VARCHAR(20)  NOT NULL                               -- con qué resultado
                      CHECK (resultado IN ('EXITOSA', 'RECHAZADA', 'FALLIDA')),
    ip_origen         VARCHAR(45)  NOT NULL,
    ocurrido_en       TIMESTAMP    NOT NULL DEFAULT NOW(),                -- cuándo
    hash_anterior     CHAR(64)     NOT NULL,                              -- huella del registro previo
```

```
    hash_actual       CHAR(64)     NOT NULL UNIQUE                        -- huella de este registro
);

-- Índices justificados por las consultas de las HU
CREATE INDEX idx_transferencia_cuenta  ON transferencia(cuenta_origen_id);            -- historial por cuenta
CREATE INDEX idx_transferencia_activas ON transferencia(estado) WHERE activo = TRUE;  -- listado sin anuladas
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria(numero_cuenta, ocurrido_en); -- HU-03: auditor
consulta por cuenta y rango de fechas (meta ≤ 5 s)
```

db/migrations/V2__bitacora_inmutable.sql

```
-- ======================================================================
-- SITRA — V2: la base de datos impide modificar o borrar la bitácora.
-- Así la regla no depende del código de la aplicación (AC-01: 0 registros modificables).
-- ======================================================================

CREATE OR REPLACE FUNCTION fn_bitacora_inmutable() RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'registro_auditoria no se puede modificar ni borrar (operación: %)', TG_OP;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bitacora_inmutable
    BEFORE UPDATE OR DELETE ON registro_auditoria
    FOR EACH ROW EXECUTE FUNCTION fn_bitacora_inmutable();
```

Cumplimiento de los requisitos evaluables

Requisito de la plantilla	Dónde se cumple
PK en toda tabla	id BIGSERIAL PRIMARY KEY en las 4 tablas.
FK con REFERENCES	transferencia → cliente_api y cuenta_autorizada; registro_auditoria → cliente_api y transferencia.
Al menos una CHECK	rol, tipo de cuenta, monto > 0, estado, operacion y resultado.
Al menos una UNIQUE	client_id, numero_cuenta, referencia_externa (evita transferencias duplicadas → 409) y hash_actual.
Borrado lógico en toda entidad de negocio	Campo activo en cliente_api, cuenta_autorizada y transferencia. Anular una transferencia = activo FALSE + estado ANULADA.
Índices justificados	idx_transferencia_cuenta (historial por cuenta); idx_transferencia_activas (listado sin anuladas); idx_auditoria_cuenta_fecha (HU-03: el auditor consulta por cuenta y rango de fechas, meta ≤ 5 s).

Excepción defendible: registro_auditoria no tiene campo activo. No es una entidad de negocio sino la evidencia del sistema: si se pudiera marcar como inactiva, dejaría de probar algo ante el auditor. La migración V2 lo hace cumplir en la propia base de datos con un trigger que rechaza cualquier UPDATE o DELETE sobre esa tabla.

Verificación ejecutada (PostgreSQL 16)

```
$ psql -d sitra -f db/migrations/V1__esquema_inicial.sql
$ psql -d sitra -f db/migrations/V2__bitacora_inmutable.sql
$ psql -d sitra -f db/pruebas/verificar_restricciones.sql
NOTICE:  OK 1: UNIQUE rechaza referencia duplicada
NOTICE:  OK 2: CHECK rechaza monto negativo
NOTICE:  OK 3: FK rechaza cuenta inexistente
NOTICE:  OK 4: CHECK rechaza estado inválido
NOTICE:  OK 5: trigger impide modificar la bitácora
NOTICE:  OK 6: trigger impide borrar la bitácora
NOTICE:  OK 7: borrado lógico conserva la transferencia y su registro de auditoría
```